Tas manuel et `std::priority_queue`

Implémentation avec `std::vector`, opérations et comparaison

100 000 · 500 000 · 1 000 000 éléments · trois répétitions

Résumé

Ce travail compare un tas max écrit manuellement avec `std::vector` et `std::priority_queue<int>`. Les deux structures reçoivent les mêmes entiers mélangés. On mesure la construction, `top()`, le retrait de tous les éléments par répétition de `pop()` et la libération du stockage. Le tas manuel expose les indices et les opérations de réorganisation ; `std::priority_queue` fournit une interface standard plus sûre et plus directement réutilisable.

1. Objectif

Un tas max garde son plus grand élément à la racine. Il est utile lorsqu'il faut accéder rapidement au prochain élément prioritaire, sans maintenir toute la séquence triée.

La comparaison porte sur deux versions équivalentes :

- un tas personnel dont les éléments sont stockés dans un `std::vector<int>` ;
- `std::priority_queue<int>`, adaptateur standard dont l'implémentation par défaut utilise un conteneur séquentiel et une relation de priorité maximale.

Les mêmes données, la même graine et les mêmes opérations sont utilisés pour éviter de comparer deux charges différentes.

2. Implémentation du tas manuel

Pour un élément à l'indice `i`, les indices du tableau sont :

Élément	Indice
Parent	`(i - 1) / 2`
Fils gauche	`2i + 1`
Fils droit	`2i + 2`

L'insertion ajoute à la fin puis remonte l'élément tant qu'il est supérieur à son parent :

```
values_.push_back(value);
sift_up(values_.size() - 1);
```

La suppression de la racine sauvegarde le maximum, place le dernier élément à la racine, réduit le vecteur avec `pop_back()`, puis descend l'élément vers le plus grand de ses fils :

```
int result = values_.front();
values_.front() = values_.back();
values_.pop_back();
if (!values_.empty())
    sift_down(0);
return result;
```

La construction bottom-up copie d'abord les valeurs puis appelle `sift_down` sur les nœuds internes, du dernier vers la racine. Elle coûte `O(n)`, contre `O(n log n)` pour insérer les `n` valeurs une par une.

3. Interface standard

La version standard est concise :

```
std::priority_queue<int> queue;
queue.push(value);
int maximum = queue.top();
queue.pop();
```

Les opérations principales sont `push`, `top`, `pop`, `empty` et `size`. `top()` est constant, tandis que `push()` et `pop()` demandent un nombre logarithmique de comparaisons. La construction depuis une séquence peut être linéaire.

Le conteneur standard masque les indices, le déplacement des éléments et la gestion de la mémoire. En contrepartie, il est moins adapté si l'on veut personnaliser la représentation interne ou observer chaque étape du sift-up et du sift-down.

4. Scénario de vidage complet

Le scénario demandé ne consiste pas à appeler une opération globale de nettoyage. Il retire réellement les éléments un par un :

```
while (!heap.empty()) {
    int value = heap.pop();
    check_priority(value);
}
```

Le benchmark vérifie que le nombre de retraits est exactement `n` et que la séquence extraite est décroissante. Le temps mesuré inclut donc tous les `sift-down` et tous les `pop_back` nécessaires.

Après ce vidage, la taille logique vaut zéro. La capacité réservée par le vecteur peut cependant rester allouée. Une seconde mesure échange le stockage avec un conteneur vide ; elle sépare donc le coût de la suppression des éléments du coût de libération du stockage.

5. Méthode expérimentale

Les valeurs `1..n` sont mélangées avec `std::mt19937` et la graine `2026`. Les tailles sont `100 000`, `500 000` et `1 000 000`. Chaque taille est mesurée trois fois pour chaque structure. La génération et le mélange sont hors chronométrage.

Les mesures sont :

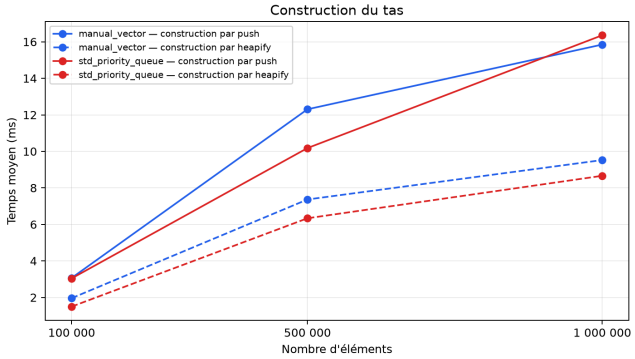
1. construction par `push` ;
2. construction bottom-up par `heapify` ;
3. `top()` appelé `n` fois sans modifier le tas ;
4. retrait des `n` éléments par `pop` ;
5. libération du stockage après le vidage.

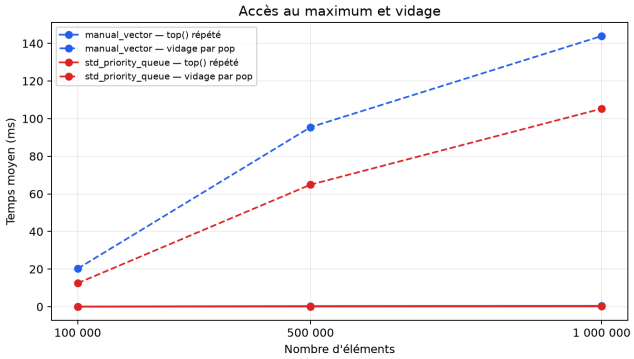
Les temps du tableau sont les moyennes des trois répétitions. Les figures sont générées par `plot_benchmarks.py` avec `uv` et `matplotlib`.

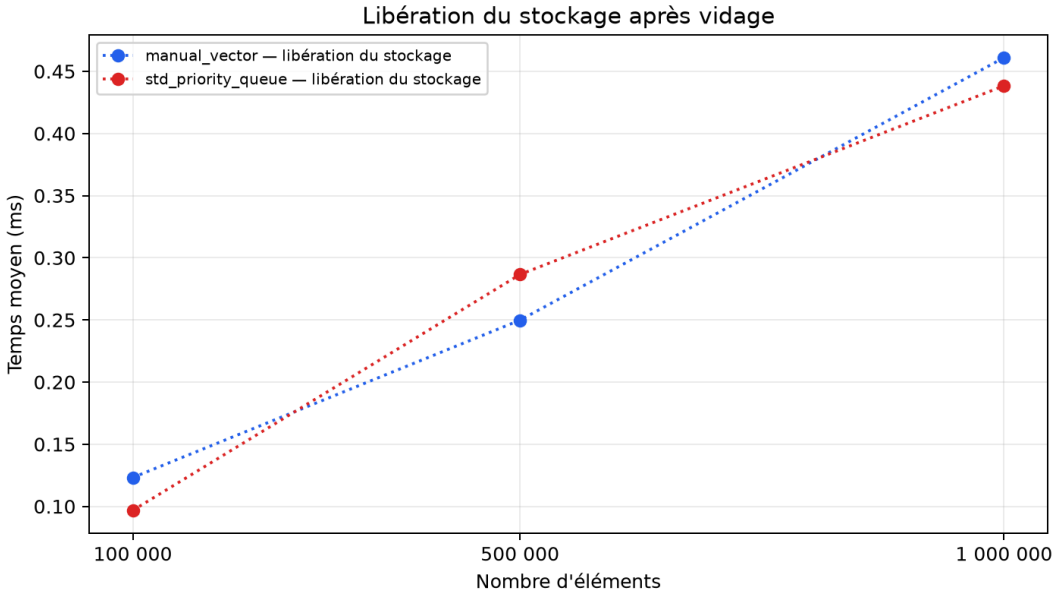
6. Résultats précédents

Les résultats ci-dessous sont produits par le fichier `benchmark.csv` et agrégés dans `report/graphs/benchmark-averages.csv`. Les temps sont en millisecondes.

n	Structure	Push	Heapify	Vidage
100 000	Tas vectoriel	3,069	1,960	20,224
100 000	`std::priority_queue`	3,044	1,498	12,501
500 000	Tas vectoriel	12,305	7,365	95,470
500 000	`std::priority_queue`	10,181	6,336	64,892
1 000 000	Tas vectoriel	15,850	9,524	143,919
1 000 000	`std::priority_queue`	16,358	8,663	105,294







À `1 000 000` éléments, `top()` répété un million de fois coûte en moyenne `0,452 ms` pour le tas vectoriel et `0,171 ms` pour `std::priority_queue`. Le vidage complet coûte respectivement `143,919 ms` et `105,294 ms`. La libération finale du stockage coûte `0,461 ms` et `0,438 ms`. Le tas vectoriel conservait une capacité de `1 048 576` cases après le vidage ; sa taille logique était déjà nulle.

7. Complexité et compromis

Opération	Complexité	Utilité
Construction par push	`O(n log n)`	Ajout progressif de priorités
Construction heapify	`O(n)`	Initialisation rapide d’un tas complet
`top()`	`O(1)`	Lire la priorité maximale
`pop()` / retrait	`O(log n)`	Traiter puis supprimer la priorité
Mémoire	`O(n)`	Stockage compact dans un vecteur

Le tas manuel est intéressant pour l’apprentissage, le contrôle de la représentation et l’ajout d’opérations spécialisées. `std::priority_queue` est préférable dans une application courante : l’interface est standard, le code est plus court et les détails de mémoire sont moins exposés aux erreurs.

Le tas ne remplace pas un tableau trié lorsqu’il faut accéder rapidement à n’importe quel rang. Il convient surtout au tri par tas, aux files de priorité, aux ordonnanceurs, aux simulations d’événements et à certains algorithmes sur les graphes.

8. Conclusion

Les deux versions ont la même idée algorithmique : garder la priorité maximale à la racine et la restaurer après chaque modification. L'implémentation avec `std::vector` montre précisément les indices et les échanges. `std::priority_queue` offre la même abstraction avec une interface plus sûre et plus pratique.

Le vidage élément par élément est une vraie suite de suppressions `O(n log n)` ; il ne doit pas être confondu avec un simple `clear`. La libération de la capacité est une étape distincte, utile pour comprendre la différence entre taille logique et mémoire réservée.

Source : `04-tas-vs-priority-queue/heap_benchmark.cpp` · graine 2026

9. Nouvelles mesures locales

Mesures effectuées à nouveau sur Intel Core i7-8665U à 1,90 GHz, 4 cœurs / 8 processeurs logiques, 14 Gio de RAM, Ubuntu Linux x86_64 (noyau 7.0.0-38-generic). Compilateur g++ 15.2.0, options -std=c++17 -O2 -Wall -Wextra -pedantic. Aucune affinité processeur imposée. Les valeurs ci-dessous remplacent les anciens résultats pour cette exécution et correspondent à benchmark-nouveau.csv.

Moyenne de trois exécutions, graine 2026 ; les temps sont en millisecondes. Chaque vidage a retiré exactement n éléments en ordre décroissant.

n	Structure	Push	Heapify	Vidage
100000	Tas manuel	2.698	1.515	17.696
100000	priority queue	2.664	1.358	12.162
500000	Tas manuel	14.306	8.498	117.187
500000	priority queue	14.524	7.397	93.158
1000000	Tas manuel	28.438	16.756	261.659
1000000	priority queue	28.771	15.073	253.380

La lecture répétée de la même racine peut être optimisée par le compilateur ; les valeurs de top ne suffisent pas à comparer les implémentations. Les mesures dépendent de la charge de la machine.